

Q1. [Asymptotic Notations]

(a)

(i)	$\Omega(n^{0.9})$	(1 mark)
(ii)	$\Omega(2^{1.01n})$	(1 mark)
(iii)	$\Omega(n!)$	(1 mark)
(iv)	$\Omega(n^2)$	(1 mark)
(v)	$\Omega(\log n)$	(1 mark)
(vi)	$\Omega(\log^2 n)$	(1 mark)

(b)

$\exists c = 2, n_0 = 1 \bullet \forall n > n_0 \bullet$ $n > 1$ $n^{0.001} > 1$ $n^{1.001} > n$ $2n < 2n^{1.001}$ $2n \leq c n^{1.001}$	(6 marks)
---	-----------

(c)

We want to prove that $[\exists c, n_0 \bullet \forall n > n_0 \bullet 2n \geq c n^{1.001}]$ is wrong. Equivalent to prove $[\forall c, n_0 \bullet \exists n > n_0 \bullet 2n < c n^{1.001}]$. $\forall c, n_0 \bullet \exists n = \max(n_0, \lceil (2/c)^{1000} \rceil) + 1 > n_0 \bullet$ $n \geq \lceil (2/c)^{1000} \rceil + 1$ $n > (2/c)^{1000}$ $n^{0.001} > 2/c$ $c n^{0.001} > 2$ $2n < c n^{1.001}$	(6 marks)
--	-----------

(d)

$\forall c \bullet \exists n_0 = \max(20000, \lceil 1/c \rceil) + 1 \bullet \forall n > n_0 \bullet$ Since $n > \lceil 1/c \rceil + 1$ $n > 1/c$ $cn > 1$ Since $n > 20001$, we have $2(n-1) \geq n, 3(n-2) \geq n, \dots, 10001(n-10000) \geq n$. Therefore, $n^{10000} < (cn)(2(n-1))(3(n-2)) \dots (10000(n-9999))(10001(n-10000))$ $n^{10000} < c(1)(2) \dots (10000)(10001)(n-10000) \dots (n-2)(n-1)(n)$ $n^{10000} < c(1)(2) \dots (10000)(10001)[(10002) \dots (n-10001)](n-10000) \dots (n-2)(n-1)(n)$ $n^{10000} < c n!$	(7 marks)
---	-----------

[Total: 25 marks]

Q2. [Solving Recurrence]

(a) Consider the recurrence $T(n)=4T(n/4)+n$.

(i)

$$\begin{aligned}
 T(n) &= 4T(n/4) + n \\
 &= 4[4T(n/4^2) + n/4] + n \\
 &= 4^2T(n/4^2) + n + n \\
 &= 4^2T(n/4^2) + 2n \\
 &= 4^2[4T(n/4^3) + n/4^2] + 2n \\
 &= 4^3T(n/4^3) + 3n \\
 &= \dots\dots \\
 &= 4^kT(n/4^k) + kn \\
 &= nT(1) + n \log_4 n, \quad \text{take } k = \log_4 n \\
 &= \Theta(n \log n)
 \end{aligned}$$

(4 marks)

(ii)

$$\begin{aligned}
 \text{Claim: } T(n) &< c n \log n, c > 1/2, n > 4 \\
 T(n) &= 4 T(n/4) + n \\
 &< 4 [c(n/4)\log(n/4)] + n \\
 &= cn \log (n/4) + n \\
 &= cn \log n - cn \log 4 + n \\
 &= cn \log n - 2cn + n \\
 &< cn \log n - n + n \\
 &= cn \log n
 \end{aligned}$$

(4 marks)

(b) Using master method to solve the following recurrences.

(i) $T(n)=4T(n/4) + n \log n$.

$$\begin{aligned}
 a &= 4, b = 4, f(n) = n \log n. \\
 \exists k = 1 \bullet f(n) &= n \log n = n^{\log_b a} \log^k n \\
 \therefore f(n) &= \Theta(n^{\log_b a} \log^k n) \\
 \text{Case 2 is applied.} \\
 T(n) &= \Theta(n^{\log_b a} \log^{k+1} n) \\
 T(n) &= \Theta(n \log^2 n)
 \end{aligned}$$

(4 marks)

(ii) $T(n)=4T(n/4) + n^{0.9} \log n$

$$\begin{aligned}
 a &= 4, b = 4, f(n) = n^{0.9} \log n. \\
 \exists \varepsilon = 0.05 \bullet f(n) &= n^{0.9} \log n < n^{0.95} = n^{1-0.05} = n^{\log_b a - \varepsilon} \\
 \therefore f(n) &= O(n^{\log_b a - \varepsilon}) \\
 \text{Case 1 is applied.} \\
 T(n) &= \Theta(n^{\log_b a}) \\
 T(n) &= \Theta(n)
 \end{aligned}$$

(4 marks)

(iii) $T(n) = 4T(n/4) + n^2 \log n$.

$$a = 4, b = 4, f(n) = n^2 \log n.$$

$$\exists \varepsilon = 1 \bullet f(n) = n^2 \log n > n^2 = n^{\log_b a + \varepsilon}$$

$$\therefore f(n) = \Omega(n^{\log_b a + \varepsilon})$$

$$\exists c = \frac{1}{4} < 1 \bullet$$

$$af\left(\frac{n}{b}\right) = 4f\left(\frac{n}{4}\right) = 4\left(\frac{n}{4}\right)^2 \log\left(\frac{n}{4}\right) = \frac{1}{4}n^2 \log\left(\frac{n}{4}\right) < \frac{1}{4}n^2 \log n = cf(n)$$

$$\therefore af\left(\frac{n}{b}\right) < cf(n)$$

Case 3 is applied.

$$T(n) = \Theta(f(n))$$

$$T(n) = \Theta(n^2 \log n)$$

(4 marks)

(c)

(i)

When master method is used for solving $T(n)$, Case 1 and Case 3 cannot coexist.

(1 mark)

(ii)

Suppose that both statements are true. So,

$$\exists \varepsilon > 0 \bullet f(n) = O(n^{\log_b a - \varepsilon}) \quad \text{and} \quad \exists \varepsilon > 0 \bullet f(n) = \Omega(n^{\log_b a + \varepsilon})$$

$$\Rightarrow \exists \varepsilon_1 > 0 \bullet \exists c_1, n_1 \bullet \forall n > n_1 \bullet f(n) \leq c_1 n^{\log_b a - \varepsilon_1} \quad \text{and} \quad \exists \varepsilon_2 > 0 \bullet \exists c_2, n_2 \bullet \forall n > n_2 \bullet f(n) \geq c_2 n^{\log_b a + \varepsilon_2}$$

$$\Rightarrow \exists \varepsilon_1 > 0, \varepsilon_2 > 0 \bullet \exists c_1, n_1, c_2, n_2 \bullet \forall n > \max(n_1, n_2) \bullet$$

$$c_2 n^{\log_b a + \varepsilon_2} \leq f(n) \leq c_1 n^{\log_b a - \varepsilon_1}$$

$$c_2 n^{\log_b a + \varepsilon_2} \leq c_1 n^{\log_b a - \varepsilon_1}$$

$$n^{\varepsilon_1 + \varepsilon_2} \leq \frac{c_1}{c_2}$$

$$\text{Contradiction is reached } \forall n > \max\left(n_1, n_2, \left(\frac{c_1}{c_2}\right)^{\frac{1}{\varepsilon_1 + \varepsilon_2}}\right).$$

(4 marks)

[Total : 25 marks]

Q3. [Binary Tree, Fundamentals of Recursive Algorithm]

The recursive definitions of the number of nodes, and the number of leaf nodes, in a binary tree T are as below:

Definition 1: If T is empty, then the number of nodes is zero. If T is not empty, then the number of nodes equals to the sum of the numbers of nodes of its left and right subtrees, plus one.

A leaf node is a node having no children nodes.

Definition 2: If T is empty, then the number of leaf nodes is zero. If $|T| = 1$, then the number of leaf nodes is one. If $|T| > 1$, then the number of leaf nodes equals to the sum of the numbers of leaf nodes of its left and right subtrees.

- (a) Based on Definition 1, give a recursive algorithm which accepts a binary tree as input, and returns its number of nodes.

```
Algorithm sum(tree)
    If tree is empty, return 0;
    return sum(left subtree) + sum(right subtree) + 1; (3 marks)
```

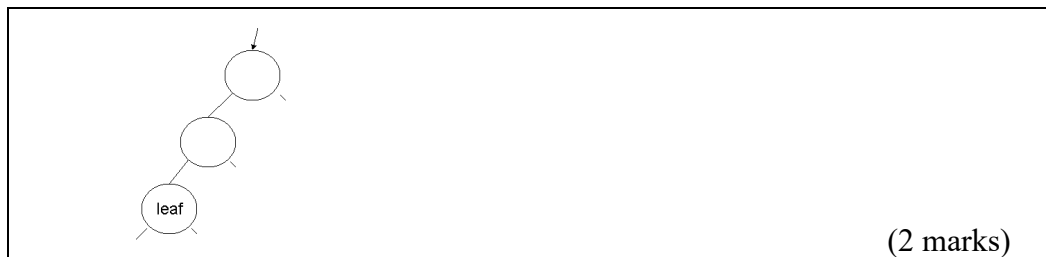
- (b) Based on Definition 2, give a recursive algorithm which accepts a binary tree as input, and returns its number of leaf nodes.

```
Algorithm sum_leaf(tree)
    If tree is empty, return 0;
    If both left and right subtrees are empty, return 1;
    return sum_leaf(left subtree) + sum_leaf(right subtree); (4 marks)
```

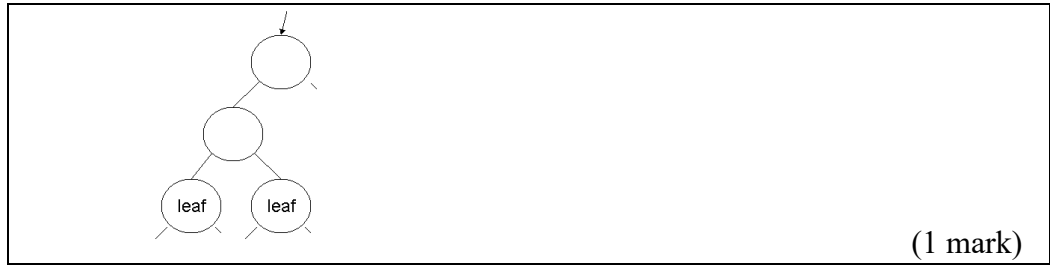
- (c) An internal node of a binary tree is a node that has at least one child node. By using the algorithms in (a) and (b), give an algorithm which accepts a binary tree as input, and returns its number of internal nodes.

```
Algorithm sum_internal(tree)
    Return sum(tree) - sum_leaf(tree); (3 marks)
```

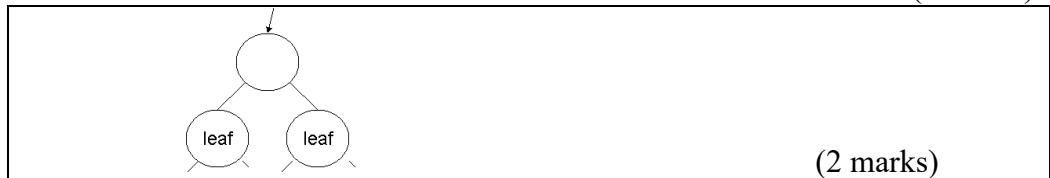
- (d) Give a non-empty binary tree such that the number of leaf nodes is less than the number of internal nodes. (2 marks)



- (e) Give a non-empty binary tree such that the number of leaf nodes is equal to the number of internal nodes. (1 mark)



- (f) Give a non-empty binary tree such that the number of leaf nodes is larger than the number of internal nodes. (2 marks)



- (g) Prove the above statement by induction on the number of internal nodes.

Base case: $\# \text{internal_node} = 1$.

The maximum number of child nodes of the only internal node is 2, and none of them is an internal node. The child nodes are leaf nodes.

The statement is true for the base case.

Assume that the statement is true for $\# \text{internal_node} = k \geq 1$.

Among all binary trees with $\# \text{internal_node} = k+1$, we choose the one with the maximum number of leaf nodes. Let L be the number of leaf nodes, and we now prove that it is $\leq k+2$.

Consider a lowest-level internal node. Its child nodes must be leaf nodes.

Let the number of the child nodes be m , and $m = 1$ or 2 . Take them away from the tree, and the internal node becomes a leaf node, and the $\# \text{internal_node} = k$. By inductive assumption, the number of leaf nodes (after trimming) is $\leq k+1$. Undo the trimming, the number of leaf nodes will be increased by $m - 1 \leq 1$. Then, the tree, before trimming, has $L \leq k+2$.

Result from induction.

(10 marks)

[Total : 25 marks]

Q4. [Design and Analysis of Algorithm]

(a)

$\Omega(n \log n)$

(1 mark)

b)

Mergesort, or heapsort

(1 mark)

(c)

$O(n \log n)$

(1 mark)

(d)

Inorder traversal

(1 mark)

(e)

Inorder traversal

(1 mark)

(f) You are given an array $A[1 .. n]$ of n integers, and one of them is x . Give an in-place $O(n)$ -time algorithm for partitioning the integers into two groups such that the first group $A[1 .. m]$ contains all non- x integers, and $A[m+1 .. n]$ contains all elements which are equal to x , for some $m \in [1, n-1]$.

[Hint: Get the idea from Hoare's algorithm.]

```
i=0; j=n+1;
while true do
    repeat j=j-1 until A[j]≠x;
    repeat i=i+1 until A[i]=x;
    if i < j
        swap A[i] and A[j];
    else
        return m; // or return, or break;
```

(5 marks)

(g) Suppose that array $A[1 .. n]$ stores n integers in arbitrary order and there are $n - \left\lceil \frac{n}{\log n} \right\rceil$ identical elements, having value x , inside the array, and the remaining $\left\lceil \frac{n}{\log n} \right\rceil$ elements have arbitrary values.

(i) Within any $2 * \left\lceil \frac{n}{\log n} \right\rceil + 1$ integers taken from $A[1 .. n]$, what is the minimum number of appearance of x ? Explain.

$\left\lceil \frac{n}{\log n} \right\rceil + 1$, because at most $\left\lceil \frac{n}{\log n} \right\rceil$ of them are the non- x integers.

(3 marks)

(ii) Give an $O(n)$ -time algorithm to find out the value of x . Explain why the time complexity is $O(n)$.

[Hint: Apply the ideas of (c), (e), and (g)(i).]

Take the integers from $A[1 \dots m]$, where $m = 2 * \left\lceil \frac{n}{\log n} \right\rceil + 1$.

Insert them into an initially-empty AVL tree.

Get the sequence of values by inorder traversal.

Among all unique values in the sequence, output the one with the highest number of occurrence. (5 marks)

- (iii) Suppose that $A[1 \dots m]$ contains all the non- x elements, where $m \in [1, n]$. Explain why $O(n)$ -time is enough to sort $A[1 \dots m]$.
[Hint: Just argue for the time complexity, and no need to go into the operations inside any sorting algorithm.]

Since there are no more than $\left\lceil \frac{n}{\log n} \right\rceil$ non- x integers, we have $m \leq \left\lceil \frac{n}{\log n} \right\rceil$. We use an optimal algorithm to sort $A[1 \dots m]$ and the time complexity is $O(m \log m) = O\left(\left\lceil \frac{n}{\log n} \right\rceil \log \left\lceil \frac{n}{\log n} \right\rceil\right) = O(n)$. (3 marks)

- (iv) By using the results in (f), (g)(ii), and (g)(iii), give an $O(n)$ -time algorithm for sorting $A[1 \dots n]$.
[Hint: If you cannot finish (f), (g)(ii), or (g)(iii), simply assume you have them done, and use them.]

Perform the algorithm in (g)(ii) and find x . – $O(n)$ time.

Perform the algorithm in (f) to partition $A[1..n]$ such that $A[1..m]$ stores the non- x integers and $A[m+1..n]$ stores value x . – $O(n)$ time.

Perform the algorithm in (g)(iii) to sort $A[1 \dots m]$. – $O(n)$ time.

If the integers stored in $A[k \dots m]$ are greater than x , where $1 \leq k \leq m$, Swap these integers with the last $m-k+1$ integers in $A[1 \dots n]$ in order.

Totally, $O(n)$ time. (4 marks)

[Total : 25 marks]